

Лабораторная работа №4

Программирование. Язык СИ. Введение в функции. Базовая работа со строками (однобайтовыми)

Комплект 1

Задание 1.1

Постановка задачи:

1.1: Создайте две функции, которые вычисляют факториал числа:

- функцию, которая вычисляет факториал, используя цикл;
- функцию, которая вычисляет факториал, используя рекурсивный вызов самой себя.

Список идентификаторов:

Имя	Тип	Смысл
fac	int	функция
fac1	int	функция

Код программы:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int fac(int x)
{
    int c = 1;
    for (int i = 1; i <= x; i++)
    {
        c = c*i;
    }
    return c;
}

int fac1(int x)
{
    if (x==1){
        return 1;
    }
    else{
        return x*fac1(x-1);
    }
}

int main()
{
    printf("%d\n", fac(10));
    printf("%d\n", fac1(10));
}

```

Результат выполненной работы:

```

3628800
3628800

```

Задание 1.2

Постановка задачи:

Объявите указатель на массив типа **int** и динамически выделите память для 12-ти элементов. Напишите функцию, которая поменяет значения чётных и нечётных ячеек массива.

Список идентификаторов:

Имя	Тип	Смысл
pa	int	указатель
i	int	Аргумент цикла
zum	void	функция

Код программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

int zum(int* s)
{
    int i = 0;
    while (s[i])
    {
        int t = s[i];
        s[i] = s[i+1];
        s[i+1] = t;
        i += 2;
    }
}

int main()
{
    int* pa;
    pa = (int*)malloc(12 * sizeof(int));

    for (int i = 0; i < 12; i++){
        scanf("%d", &pa[i]);
    }
    zum(pa);
    for (int i = 0; i < 12; i++){
        printf("%d\t", pa[i]);
    }
    free(pa);
    return 0;
}
```

Результат выполненной работы:

1	2	3	4	5	6	7	8	9	10	11	12																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
---	---	---	---	---	---	---	---	---	----	----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Задание 1.3

Постановка задачи:

Создать две основные функции:

- функцию для динамического выделения памяти под двумерный динамический массив типа **double** — матрицу;
- функцию для динамического освобождения памяти под двумерный динамический массив типа **double** — матрицу.

Создать две вспомогательные функции:

- функцию для заполнения матрицы типа **double**;
- функцию для распечатки этой матрицы на экране.

Продемонстрировать работу всех этих функций в своей программе.

Список идентификаторов:

Имя	Тип	Смысл
All_matr	double	функция
fill_matr	void	функция
pr_matr	void	функция
fr_matr	void	функция
n	int	Входной параметр
m	int	Входной параметр
pa	double	Указатель

Код программы:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

double** all_matr(int n, int m)
{
    double** A = (double**)malloc(n * sizeof(double*));
    for (int i = 0; i < n; i++){
        A[i] = (double*)malloc(m*sizeof(double));
    }
    return A;
}

void fr_matr(double** A, int n)
{
    for (int i=0; i < n; i++){
        free(A[i]);
    }
    free(A);
}

void fill_matr(double** A, int n, int m)
{
    for (int i=0; i<n; i++){
        for (int j = 0; j<m;j++){
            printf("enter A[%d][%d]: ", i, j);
            scanf("%lf", &A[i][j]);
        }
    }
    printf("\n");
}

void pr_matr(double** A, int n, int m)
{
    for (int i=0; i<n; i++){
        for (int j = 0; j<m;j++){
            printf("%2lf\t", A[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int n, m;
    printf("enter the number of rows and columns: ");
    scanf("%d%d", &n, &m);

    double** pa = all_matr(n, m);
    fill_matr(pa, n, m);
    pr_matr(pa, n ,m);
    fr_matr(pa, n);

    return 0;
}

```

Результат выполненной работы:

```

enter the number of rows and columns: 3 3
enter A[0][0]: 1
enter A[0][1]: 2
enter A[0][2]: 3
enter A[1][0]: 4
enter A[1][1]: 5
enter A[1][2]: 6
enter A[2][0]: 7
enter A[2][1]: 8
enter A[2][2]: 9

1.000000      2.000000      3.000000
4.000000      5.000000      6.000000
7.000000      8.000000      9.000000

```

Задание 4.4

Постановка задачи:

Создать функцию, которая вычисляет векторное произведение двух векторов в декартовых координатах, используя указатели на соответствующие массивы.

Список идентификаторов:

Имя	Значение	Тип
All_matr	функция	int
Det_matr	Функция	Double
Pr_matr	Функция	Void
Fr_matr	Функция	Void
a	Массив	Int
b	Массив	Int
pa	Указатель	int

Код программы:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

int** all_matr(int* a, int* b)
{
    int** A = (int**)malloc(3 * sizeof(int*));
    for (int i = 0; i < 3; i++){
        A[i] = (int*)malloc(3 * sizeof(int*));
    }
    A[0][0] = 1; A[0][1] = 1; A[0][2] = 1;
    A[1] = a;
    A[2] = b;

    return A;
}

void fr_matr(int** A)
{
    for (int i=0; i<3; i++){
        free(A[i]);
    }
    free(A);
}

void pr_matr(int** A)
{
    for (int i=0; i <3; i++){
        for (int j = 0; j<3; j++){
            printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
}

```

```

double det_matr(int** A)
{
    return((A[0][0]*A[1][1]*A[2][2]) + (A[1][0]*A[2][1]*A[0][2]) + (A[2][0]*A[0][1]*A[1][2])) - ((A[2][0]*A[1][1]*A[0][2]) + (A[0][1]*A[1][0]*A[2][2]) + (A[0][0]*A[2][1]*A[1][2]));
}

int main()
{
    int* a = (int*)malloc(3 * sizeof(int));
    printf("enter 1 [x y z]: ");
    scanf("%d%d%d", &a[0], &a[1], &a[2]);

    int* b = (int*)malloc(3 * sizeof(int));
    printf("enter 2 [x y z]: ");
    scanf("%d%d%d", &b[0], &b[1], &b[2]);

    int** pa = all_matr(a,b);
    pr_matr(pa);
    printf("\n");
    printf("vector = %.2lf\n", det_matr(pa));

    fr_matr(pa);

    return 0;
}

```

Результат выполненной работы:

```
enter 1 [x y z]: 3 3 1
enter 2 [x y z]: 7 8 9
1      1      1
3      3      1
7      8      9

vector = 2.00
```

Комплект 2.

Задание 2.1

1.

Постановка задачи:

Вычислите длину строки my_string, используя цикл for

Код программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define size 20

int main()
{
    char* string = (char*)malloc(size * sizeof(char));
    fgets(string, size, stdin);

    int count = 0;
    for (int i = 0; string[i] != '\0'; i++){
        count += 1;
    }

    printf("%d", count - 1);

    free(string);

    return 0;
}
```

Результат выполненной работы:

```
i love cats
11
```

3.

Постановка задачи:

Используйте функции

size_t strlen(const char* str); или

3

size_t strnlen (const char *string, size_t maxlen); или

size_t strnlen_s(const char *str, size_t strsz); для получения размера строки в виде значения

Код программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define size 20

int main()
{
    char* string = (char*)malloc(size * sizeof(char));
    fgets(string, size, stdin);

    printf("%d", strlen(string));

    free(string);

    return 0;
}
```

Результат выполненной работы:

```
i love cats
11
```

4.

Постановка задачи:

Создайте вторую строку (второй массив) и скопируйте в неё строку my_string, используя функцию

char *strcpy(char *dest, const char *src); или

char *strncpy (char *dest, const char *src, size_t n);

Код программы:

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define size 20

int main()
{
    char* string = (char*)malloc(size * sizeof(char));
    fgets(string, size, stdin);

    char* nstring = (char*)malloc(size * sizeof(char));
    fgets(nstring, string);

    printf("\n%s\n%s\n", string, nstring);

    free(string);
    free(nstring);

    return 0;
}

```

Результат выполненной работы:

```

love cats
love cats
love cats

```

Задание 2.2

Постановка задачи:

Конвертируйте введенные заданные как строки: число с плавающей

точкой (double) и целое число (int) в значения типа double и int, используя функции atof и atoi

Список идентификаторов:

Имя	Значение	Тип
size	константа	const
string	Указатель	char
nstring	указатель	char

Код программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define size 20

int main()
{
    char* string = (char*)malloc(size * sizeof(char));
    fgets(string, size, stdin);

    char* nstring = (char*)malloc(size * sizeof(char));
    fgets(nstring, string);

    printf("\n%.2lf\n", atof(string));
    printf("%d\n", atoi(nstring));

    free(string);
    free(nstring);

    return 0;
}
```

Результат выполненной работы:

```
123.45
6789

123.45
6789
```

ФИО:

Закаблукова Анастасия Эдуардовна, ИВТ 1 курс, группа 1.1